threads are scheduled in a particular fashion, then the more times the code is run, the more likely the bug is to appear. If you run the test once and it passes, you might feel a bit of confidence that the code works. If you run it ten times in a row and it passes every time, you'll likely feel more confident. If you run the test a billion times and it passes every time, you'll feel more confident still.

The confidence you have in the results does depend on the amount of code being tested by each test. If your tests are quite fine-grained, like the tests outlined previously for a thread-safe queue, such brute-force testing can give you a high degree of confidence in your code. On the other hand, if the code being tested is considerably larger, the number of possible scheduling permutations is so vast that even a billion test runs might yield a low level of confidence.

The downside to brute-force testing is that it might give you false confidence. If the way you've written the test means that the problematic circumstances can't occur, you can run the test as many times as you like and it won't fail, even if it would fail *every* time in slightly different circumstances. The worst example is where the problematic circumstances can't occur on your test system because of the way the particular system you're testing on happens to run. Unless your code is to run only on systems identical to the one being tested, the particular hardware and operating system combination may not allow the circumstances that would cause a problem to arise.

The classic example here is testing a multithreaded application on a single-processor system. Because every thread has to run on the same processor, everything is automatically serialized, and many race conditions and cache ping-pong problems that you may get with a true multiprocessor system evaporate. This isn't the only variable though; different processor architectures provide different synchronization and ordering facilities. For example, on x86 and x86-64 architectures, atomic load operations are always the same, whether tagged `memory_order_relaxed` or `memory_order_seq_cst` (see section 5.3.3). This means that code written using relaxed memory ordering may work on systems with an x86